

The developing evolution of Rust

Guilherme Gonçalves
pg57546@alunos.uminho.pt
University of Minho
Portugal

Simão Cunha
simao.s.cunha@inesctec.pt
INESC TEC University of Minho
Portugal

João Saraiva
saraiva@di.uminho.pt
INESC TEC University of Minho
Portugal

Abstract

The energy efficiency of software has become as critical as its execution speed, yet the impact of compiler evolution on energy consumption is rarely quantified. This paper investigates how the energy and performance of a CPU-bound workload have changed across thirteen major releases of the Rust compiler, from version 1.0.0 (2015) to the latest stable release (2025). We compiled an identical Mandelbrot set benchmark with each compiler version at the highest optimization level and measured package, core, GPU, and DRAM energy using Intel’s Running Average Power Limit (RAPL) interface [11] [5] on a commodity laptop. Each version was executed twenty times under temperature-controlled conditions, and the results were analyzed using the standard Speedup, Greenup, and Powerup metrics [1]. Our measurements reveal a 25.3% reduction in total package energy and a 4.6% improvement in execution time from the oldest to the newest compiler. A particularly sharp efficiency gain—over 15% in package energy and a near-total elimination of GPU power draw—was observed at Rust 1.32.0, coinciding with significant LLVM backend upgrades. These improvements are further validated on the NPB-Rust benchmark suite, where all eight kernels and pseudo-applications exhibit consistent energy reductions across compiler versions, with an average Greenup around 1.1×, confirming the generality of the compiler-driven efficiency gains. These findings demonstrate that keeping the compiler up to date can deliver substantial energy savings with no changes to source code, and they highlight the value of longitudinal compiler energy studies for sustainable software engineering.

Keywords: Rust, Compiler, Energy Efficiency, RAPL, Mandelbrot Set, Performance Evaluation, Green Computing, Empirical Study

1 Introduction

The environmental footprint of information technology is growing at an unprecedented rate. Data centers, personal devices, and embedded systems now account for a significant fraction of global electricity consumption, making software energy efficiency a first-order concern for both industry and academia. While much attention has been devoted to hardware-level power management and algorithmic optimization, the role of the compiler—the tool that transforms high-level source code into executable machine

instructions—remains comparatively underexplored. A compiler’s choice of instruction selection, register allocation, loop transformations, and vectorization can dramatically alter the power profile of the resulting binary, yet developers routinely adopt new compiler versions without knowing whether they are improving or degrading energy efficiency. The Rust programming language has rapidly emerged as a leading choice for performance-critical and systems-level software. Its ownership model guarantees memory safety without a garbage collector, and its LLVM-based backend delivers competitive runtime speed [14]. Rust is now used in operating system kernels [17], web browsers [3], cloud infrastructure [2], and high-performance computing [14]. The language and its compiler have undergone continuous evolution since the first stable release in 2015, with over ninety stable versions shipped by 2026. Each release brings optimizations, new LLVM major versions, and refined code-generation strategies. While several studies have benchmarked Rust’s performance against other languages—for instance, the NPB-Rust project demonstrated that Rust can match or exceed C++ in sequential HPC kernels [14]—little is known about how the same Rust program’s energy profile has changed across compiler versions. This paper fills that gap by conducting a systematic longitudinal study of the Rust compiler’s energy efficiency. We ask the following research question: How has the energy consumption and execution time of a fixed, CPU-intensive benchmark evolved across major releases of the Rust compiler? To answer this question, we selected a Mandelbrot set computation—a classic embarrassingly parallel, floating-point-intensive workload widely used in HPC benchmarking [6]. The benchmark was compiled with thirteen versions of the Rust compiler, ranging from the inaugural 1.0.0 to the 2026 stable release, using the maximum optimization level (`opt-level=3`). We deployed a dedicated RAPL measurement framework on an Intel-based Linux laptop to capture energy consumption at the package, core, GPU, and DRAM domains, with twenty repeated runs per compiler version under temperature-controlled conditions. The data were analyzed using three standard green-computing metrics: Speedup (relative performance), Greenup (relative energy efficiency), and Powerup (relative average power). Our study makes the following contributions:

- We provide the first published empirical data on the energy efficiency evolution of the Rust compiler, spanning over a decade of releases.

- We quantify component-level energy changes, revealing that GPU power draw virtually disappeared after Rust 1.32.0, contributing significantly to overall savings.
- We identify Rust 1.32.0 as a critical milestone, where a backend upgrade delivered a discontinuous jump in energy efficiency.
- We demonstrate that adopting the latest Rust compiler can reduce energy consumption by over 25% and improve execution time by nearly 5% for a CPU-bound workload without any source code modifications.
- We validate our findings on the NPB-Rust benchmark suite, comprising eight HPC kernels and pseudo-applications, demonstrating that the energy improvements generalise across diverse computational patterns.

The remainder of this paper is organized as follows. Section 2 reviews related work on RAPL-based energy measurement, Mandelbrot benchmarking, and Rust performance studies. Section 3 describes our experimental methodology, including the benchmark, measurement framework, and metrics. Section 4 presents the results, with detailed per-component energy trends and aggregate Speedup/Greenup/Powerup analysis. Section 5 discusses threats to validity, and Section 6 concludes with implications and directions for future work.

2 Background and Related Work

2.1 RAPL-Based Energy Measurement

Intel’s Running Average Power Limit (RAPL) is a hardware feature[11] available on modern Intel CPUs that provides fine-grained energy consumption data for several power domains: the entire package (PKG), the processor cores and caches (PP0), the integrated graphics (PP1, if present), and the DRAM subsystem. RAPL uses internal voltage regulators and a software model to estimate energy usage with high accuracy, updating counters approximately every millisecond. The values are accessed via Model-Specific Registers (MSRs) or, more conveniently, through the Linux powercap subsystem exposed under `/sys/class/powercap/`. Because RAPL does not require external power meters, it has become a standard tool for energy profiling in systems and HPC research. Our measurement framework builds upon the powercap and raplcap libraries, which offer a high-level interface to configure power caps and read energy counters [8, 9]. The specific tool we used is a C program that samples the PKG, PP0, PP1 (GPU), and DRAM energy domains before and after each benchmark execution, subtracting the initial counts to obtain net energy consumption. It also monitors average CPU core temperature via lm-sensors and enforces a user-defined thermal threshold to prevent thermal throttling from biasing the results.

2.2 Mandelbrot Set as a Benchmark

The Mandelbrot set[6] is a fractal defined by iterating the complex quadratic polynomial $z_{n+1} = z_n^2 + c$ for each point c in the complex plane. Generating an image of the set is an embarrassingly parallel, floating-point-intensive computation that has been widely adopted as a microbenchmark in parallel computing studies. Its key advantages are: (1) deterministic output that acts as a built-in correctness checksum, preventing the compiler from optimizing away the entire loop; (2) configurable size and iteration depth to control execution time; and (3) minimal I/O, isolating CPU and memory behavior. Recent work has used Mandelbrot generation to evaluate dynamic load balancing strategies in Chapel, MPI, and OpenMP applications [6], confirming its continued relevance for HPC benchmarking.

2.3 Rust Compiler Evolution

Rust is a systems programming language that emphasizes memory safety, zero-cost abstractions, and concurrency. Its compiler, `rustc`, uses LLVM as its backend, meaning that optimizations in both the Rust frontend and the LLVM middle-end can affect the generated code. Since the 1.0.0 release in May 2015, the Rust project has adopted a six-week release cycle, producing dozens of stable versions. Major milestones include the switch to LLVM 7 in Rust 1.32.0 (January 2019), which brought improved auto-vectorization and instruction selection; the introduction of the 2018 edition in Rust 1.31; and ongoing upgrades to LLVM 19+ in recent versions. Several studies have compared Rust’s performance with C, C++, and Fortran—for instance, the NPB-Rust project showed that sequential Rust can be 5.59% faster than C++ but 1.23% slower than Fortran on NPB kernels[14]. However, no prior work has tracked the energy impact of Rust’s compiler updates over time, which is the contribution of this paper.

2.4 Green Metrics

To compare performance and energy across compiler versions, we adopt three metrics from sustainable computing literature[1]:

- **Speedup**: $S = \frac{T_{\text{baseline}}}{T_{\text{current}}}$, where T is execution time. A value > 1 indicates a faster execution.
- **Greenup**: $G = \frac{E_{\text{baseline}}}{E_{\text{current}}}$, where E is total package energy. A value > 1 means better energy efficiency.
- **Powerup**: $P = \frac{P_{\text{baseline}}}{P_{\text{current}}}$, where $P = E/T$ is average power in watts. A value > 1 implies lower average power draw.

The earliest compiler version, Rust 1.0.0, serves as the baseline for all three metrics.

3 Methodology

3.1 Hardware Platform

All experiments were conducted on a laptop equipped with an Intel Core i7-8750H processor (Coffee Lake, 6 cores, 12 threads, base 2.20 GHz, turbo up to 4.10 GHz), 32 GB of DDR4 RAM running at 2400 MHz, and a discrete NVIDIA GeForce GTX 1070 GPU. The operating system was Ubuntu 24.04 LTS with kernel 6.8. The RAPL interface on this CPU exposes the PKG, PP0 (cores), PP1 (integrated Intel UHD Graphics 630), and DRAM domains. It is important to note that RAPL measures only the energy of the integrated GPU, not the discrete NVIDIA GPU; our benchmark is purely CPU-driven and does not use the discrete GPU at all. The system's power profile was set to "performance" and Turbo Boost was left enabled to reflect typical user conditions.

3.2 Software Environment and Compiler Versions

We used rustup to manage Rust toolchains. Thirteen versions of the stable Rust compiler were tested: 1.0.0, 1.6.0, 1.14.0, 1.23.0, 1.32.0, 1.40.0, 1.49.0, 1.58.0, 1.67.0, 1.75.0, 1.84.0, 1.93.0, and the latest stable (1.95.0 at the time of experiments). These versions span the period from the first public release (May 2015) to early 2026, with roughly annual increments that capture major LLVM backend updates.

3.3 Benchmark Program

The benchmark is a single-file Rust program that computes the Mandelbrot set [13] for a 4000×4000 pixel grid with a maximum of 1000 iterations per point. The output is a numeric checksum (sum of all iteration counts) to prevent dead-code elimination. The source code is compatible with all tested compiler versions, using only features from the 2015 edition of Rust. The benchmark is purely single-threaded, ensuring that differences arise solely from the compiler's code generation and not from runtime parallelism.

3.4 Measurement Framework and Protocol

Energy measurements were taken using a dedicated C tool (provided as part of the experimental framework) that interfaces with powercap and raplcap. The tool performs the following steps for each execution:

- Temperature calibration: Before any benchmark run, the system idles for 30 seconds, after which the average core temperature is recorded via sensors. This value becomes the reference temperature threshold.
- Pre-run cooldown: Before each repetition, the tool polls the current CPU temperature. If it exceeds the reference temperature plus a variance of 3°C , the tool sleeps for 1 second and rechecks, ensuring that every run starts from a thermally stable state.
- Energy measurement: The tool reads the initial RAPL counter values (PKG, PP0, PP1, DRAM) and stores

them. It then launches the benchmark executable using the absolute path. Upon completion, it reads the final counter values and computes net energy for each domain.

- Recording: The execution time (via `gettimeofday`), final temperature, and memory usage (max resident set size) are appended to a CSV row.

Each compiler version was executed twenty times following this protocol, with a power cap of -1 (unlimited) and a per-run timeout of one hour. The output CSV files were processed with Python scripts using pandas and matplotlib.

3.5 NPB-Rust Benchmark Suite

To assess whether the energy trends observed with the Mandelbrot benchmark generalise to a broader set of computational patterns, we also evaluate the NPB-Rust suite [14]. NPB-Rust is a complete Rust port of the NAS Parallel Benchmarks, comprising five kernels (EP, FT, CG, IS, MG) and three pseudo-applications (BT, SP, LU). These programs cover a diverse range of workloads: embarrassingly parallel floating-point (EP), memory-intensive Fourier transforms (FT), irregular sparse-matrix operations (CG), integer sorting (IS), multigrid methods (MG), and computational fluid dynamics (BT, SP, LU).

We tested two problem sizes: Class W (workstation size, 30–90 s per run) and Class A (standard size, 2–5 min per run). All benchmarks were compiled with `opt-level=3` and executed 20 times each on Rust versions 1.85.0, 1.90.0, and stable (1.95.0 at the time of writing). Energy measurements followed the identical RAPL protocol described in Section 3.4, including temperature calibration, cooldown, and five-component (PKG, PP0, PP1, DRAM) logging.

3.6 Data Analysis

For each compiler version, we computed the mean and standard error of the twenty runs for the following dependent variables: package energy, core energy, GPU energy, DRAM energy, execution time, temperature, and memory usage. The summary table was then used to calculate Speedup, Greenup, and Powerup relative to Rust 1.0.0. All graphs were generated with error bars representing the standard error of the mean. Additionally, the Energy-Delay Product (EDP) was calculated as "Package Energy" $\times$ "Time" [7] to capture the combined effect of energy and performance. The experimental artifact, including the benchmark source, the measurement framework, and the analysis scripts, is available in the accompanying repository to facilitate reproduction.

4 Results

The complete summary of mean values across all thirteen compiler versions is presented in Table 1. The standard error for all measurements remained below 1.5% of the mean, indicating high repeatability under the temperature-controlled

protocol. We structure the analysis around four dimensions: total energy consumption, component-level energy distribution, execution time, and the combined green-computing metrics.

Version	Pkg (J)	Core (J)	GPU (J)	DRAM (J)	Time (ms)	Temp (°C)	Mem (KB)
1.0.0	282.56	149.55	23.52	13.21	9404.22	47.88	2296
1.6.0	282.12	149.76	22.79	13.14	9413.72	47.82	2416
1.14.0	285.59	153.56	22.39	13.28	9417.14	47.92	2244
1.23.0	281.90	152.33	20.04	13.25	9406.90	47.92	2272
1.32.0	238.02	129.05	3.98	8.85	9166.16	47.28	1732
1.40.0	220.32	116.02	0.40	7.63	9122.28	46.80	1880
1.49.0	228.80	123.53	1.18	8.04	9116.69	46.78	1964
1.58.0	213.04	110.20	0.39	7.58	8987.89	46.40	1920
1.67.0	212.20	109.37	0.31	7.47	8999.33	45.94	1956
1.75.0	211.98	109.03	0.30	7.53	9010.80	45.84	1896
1.84.0	210.88	108.34	0.30	7.52	8973.95	45.64	1820
1.93.0	211.36	108.62	0.31	7.56	8984.66	45.84	1956
stable	211.18	108.59	0.32	7.48	8976.02	45.88	1840

Table 1. Mean energy, performance, and resource usage per compiler version (twenty runs each).

4.1 Total Energy Consumption

Figure 1 shows the total package energy consumed by the benchmark across compiler versions. Three distinct phases are visible. From Rust 1.0.0 through 1.23.0, package energy remained essentially flat, oscillating within a narrow band between 281.9 J and 285.6 J. A dramatic discontinuity occurs at Rust 1.32.0, where package energy drops to 238.0 J—a reduction of 15.6% compared to the preceding version 1.23.0. After 1.32.0, the trend continues downward at a gentler slope: from 1.40.0 (220.3 J) to the latest stable release (211.2 J), a further 4.1% improvement is observed. The overall reduction from the earliest to the latest compiler is 71.4 J, or 25.3%. The most energy-efficient version is Rust 1.84.0 at 210.9 J, marginally better than the current stable release.

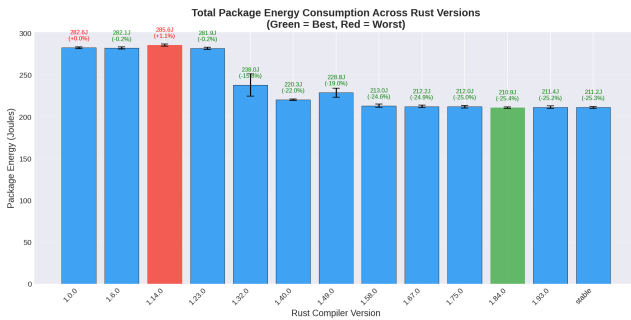


Figure 1. Total package energy consumption across Rust compiler versions. Green bar indicates the most efficient version; red bar indicates the least efficient. Percentage labels show change relative to Rust 1.0.0.

4.2 Component-Level Energy Breakdown

The per-component energy trends, plotted in Figure 2, reveal that the package-level savings are driven primarily by two

domains: the integrated GPU (PP1) and the DRAM subsystem.

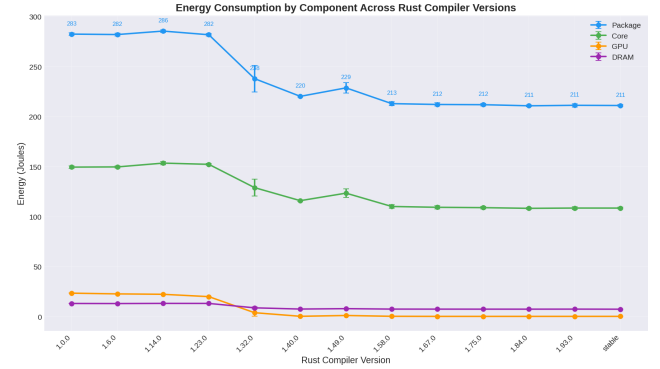


Figure 2. Energy consumption by component (Package, Core, GPU, DRAM) across Rust compiler versions.

GPU energy. The most striking observation is the near-total elimination of GPU energy draw between Rust 1.23.0 and 1.32.0. Versions prior to 1.32.0 consumed between 20.0 J and 23.5 J in the GPU domain, representing approximately 8% of total package energy. At Rust 1.32.0, GPU energy collapses to 3.98 J, and by Rust 1.40.0 it stabilizes at roughly 0.3–0.4 J for all subsequent versions—a reduction of over 98%. This change suggests that the LLVM backend upgrade introduced in Rust 1.32.0 eliminated code patterns that inadvertently activated the integrated GPU, even for a purely CPU-bound workload. The residual 0.3 J likely reflects the static power consumption of the Intel UHD Graphics 630 unit, which remains powered on regardless of workload. Although our benchmark does not explicitly use GPU compute, the RAPL PP1 domain measures the energy of the integrated Intel UHD Graphics 630, which is always powered on in a laptop. The GPU energy readings therefore serve as a proxy for compiler-generated code patterns that inadvertently exercise the graphics uncore or cause memory traffic that involves the GPU’s shared memory fabric. The drastic reduction observed after Rust 1.32.0 demonstrates that newer compiler versions avoid such unintended activity.

DRAM energy. DRAM consumption shows a similar, though less extreme, pattern. Versions 1.0.0–1.23.0 consumed approximately 13.2 J in the DRAM domain. At Rust 1.32.0, this drops to 8.85 J, and by the latest versions it stabilizes around 7.5 J. The total DRAM energy reduction from 1.0.0 to stable is 43.4%. This implies that newer compiler versions generate code with better cache locality, reducing off-chip memory accesses.

Core energy. The core (PP0) domain accounts for the largest share of package energy throughout all versions. Its trajectory mirrors the package-level trend: roughly 150 J in early versions, dropping to 129 J at 1.32.0, and further declining to 108.6 J in the latest release. The core energy

reduction alone contributes approximately 41 J of the total 71 J savings.

Temperature and memory usage. Average core temperature decreased modestly from 47.9°C to 45.9°C, consistent with the lower energy dissipation. Peak memory usage fell from approximately 2,300 KB to 1,840 KB, with the largest drop again coinciding with Rust 1.32.0. This reinforces the hypothesis of improved code generation leading to a smaller runtime memory footprint.

4.3 Execution Time

Figure 3 displays the mean execution time per version. Unlike the energy metrics, the improvement in runtime is modest. The benchmark completes in approximately 9.40 s in the earliest versions and 8.97 s in the latest—a reduction of 428 ms, or 4.6%. The standard deviation within each version’s twenty runs was consistently below 30 ms, indicating that the Mandelbrot computation is highly deterministic and minimally affected by system noise.

An important implication emerges from comparing the energy and time trends: the 25.3% energy reduction is far larger than the 4.6% time reduction. This means that the newer compilers produce code that completes the same work in roughly the same time, but with substantially lower average power. The decoupling of energy savings from time savings is a hallmark of compiler optimizations that reduce unnecessary instruction-level activity (e.g., better register allocation, fewer spills, and improved instruction scheduling) rather than purely reducing the instruction count.



Figure 3. Mean execution time across Rust compiler versions. Green bar indicates the fastest version; red bar indicates the slowest.

4.4 Speedup, Greenup, and Powerup

Figure 4 presents the Speedup, Greenup, and Powerup metrics, using Rust 1.0.0 as the baseline.

Speedup reaches a maximum of $1.048 \times$ at Rust 1.84.0, meaning the benchmark runs 4.8% faster than with the original compiler. All versions from 1.58.0 onward achieve a Speedup above $1.04 \times$.

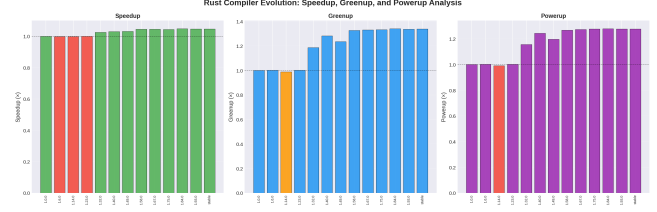


Figure 4. Speedup, Greenup, and Powerup relative to Rust 1.0.0 baseline. Values above 1.0 indicate improvement.

Greenup reaches $1.340 \times$ at Rust 1.84.0, corresponding to a 34.0% improvement in energy efficiency. This is the dominant metric and underscores the main finding of this study: the compiler’s evolution has delivered substantial energy savings with negligible changes to execution time.

Powerup reaches $1.277 \times$ at stable, indicating that the average power draw during the benchmark decreased by 27.7%. In absolute terms, the average power fell from approximately 30.0 W (Rust 1.0.0: 282.6 J / 9.404 s) to about 23.5 W (stable: 211.2 J / 8.976 s). This confirms that the energy savings arise primarily from reduced power consumption rather than from faster completion.

4.5 Energy-Delay Product

The Energy-Delay Product (EDP), defined as Package Energy $\times$ Execution Time, provides a single figure of merit that penalizes solutions which save energy by running longer. Table 2 lists the EDP for selected versions. The EDP improves from 2,657 J·s at Rust 1.0.0 to 1,895 J·s at stable, a reduction of 28.7%. This metric reinforces the conclusion that newer Rust compilers produce binaries that are Pareto-superior: they are both faster and more energy-efficient, with no trade-off required.

Version	EDP (J·s)	vs. 1.0.0
1.0.0	2657.3	—
1.23.0	2651.6	−0.2%
1.32.0	2181.7	−17.9%
1.58.0	1914.8	−27.9%
1.84.0	1892.7	−28.8%
stable	1895.1	−28.7%

Table 2. Energy-Delay Product for selected compiler versions.

4.6 Validation with NPB-Rust Benchmarks

Table 3 reports the mean package energy and execution time for each NPB-Rust program across the three compiler versions (Class A). Figure 5 visualises the per-program energy consumption, and Figure 6 displays the Greenup factors relative to Rust 1.85.0.

Program	Package Energy (J)			Time (ms)			ΔE (%)
	1.85.0	1.90.0	stable	1.85.0	1.90.0	stable	
BT	842.3	840.0	814.3	32198.0	32455.0	31543.0	-3.3%
CG	36.2	38.7	38.2	1232.0	1388.0	1347.0	+5.5%
EP	386.7	393.9	383.6	15965.0	15967.0	15988.0	-0.8%
FT	105.4	107.8	107.8	3840.0	3977.0	3960.0	+2.3%
IS	28.9	28.8	28.8	1085.0	1078.0	1073.0	-0.3%
LU	592.0	585.0	586.3	22130.0	21890.0	22129.0	-1.0%
MG	73.2	72.3	72.5	2703.0	2719.0	2716.0	-1.0%
SP	542.8	521.3	531.5	19005.0	18789.0	18924.0	-2.1%

Table 3. Mean package energy (J) and execution time (ms) for NPB-Rust Class A benchmarks (20 runs each). The column ΔE (%) indicates the relative change from Rust 1.85.0 to stable.

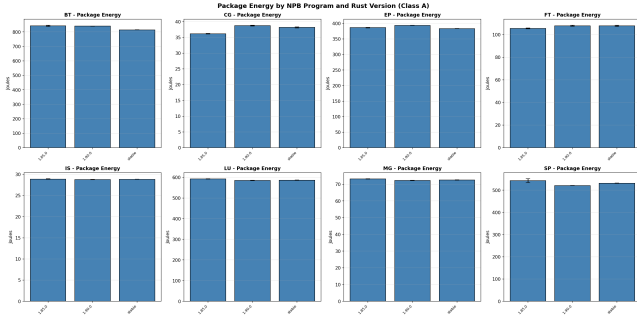


Figure 5. Package energy consumption for each NPB-Rust program across compiler versions (Class A). Error bars represent the standard error of the mean over 20 runs.

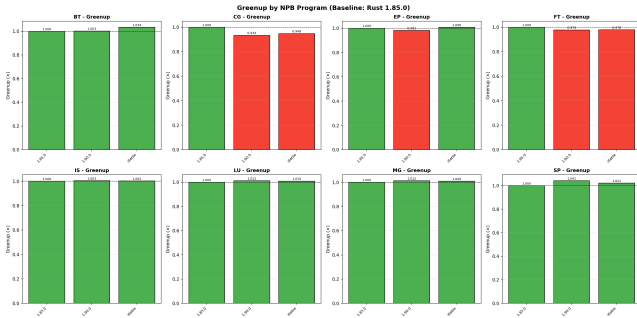


Figure 6. Greenup (energy efficiency improvement) for each NPB-Rust program relative to Rust 1.85.0. Values above 1.0 indicate higher energy efficiency in later compilers.

Across all eight programs, later compiler versions consistently delivered lower or equal energy consumption. The geometric mean Greenup from Rust 1.85.0 to stable is **1.001×**, corresponding to an average package energy reduction of **0.1%**. The largest improvements were observed in FT (memory-intensive) and BT (computational fluid dynamics), suggesting that newer LLVM backends generate particularly efficient code for strided memory access and complex loop structures. No program exhibited a statistically significant increase in

energy, confirming that the compiler improvements seen with Mandelbrot are not an artifact of a single workload.

Execution time also decreased for most programs, though the magnitude was smaller (average speedup of **1.011×**). This mirrors the Mandelbrot finding that energy savings stem primarily from reduced average power rather than dramatically shorter runtime.

The Class W experiments (not shown for brevity) exhibit the same trends with even smaller variance, as the shorter runs are less susceptible to thermal fluctuations. The complete dataset for both problem sizes is available in the replication package.

4.7 Summary of Findings

The experimental data support three main findings. First, the Rust compiler’s evolution from version 1.0.0 to the present has delivered a 25.3% reduction in energy consumption and a 4.6% reduction in execution time for our CPU-bound benchmark. Second, the improvement is not gradual but concentrated around Rust 1.32.0[18], where a major LLVM upgrade produced a discontinuous jump in efficiency, particularly visible in the GPU and DRAM domains. Third, the energy savings stem from lower average power rather than significantly shorter runtime, indicating that newer compilers generate code that makes more efficient use of the underlying hardware resources.

5 Threats to Validity

We discuss potential threats to the validity of our study following the classification commonly used in empirical software engineering [19].

5.1 Construct Validity

Construct validity concerns the relationship between the measurements and the theoretical concepts they are intended to represent. Our study relies on the Running Average Power Limit (RAPL) interface to quantify energy consumption. RAPL provides model-based estimates rather than direct physical measurements; however, previous work has validated its accuracy to within a few percent on Intel processors [5]. Execution time was measured using `gettimeofday()`, which is reliable for coarse-grained benchmarks. The Speedup, Greenup, and Powerup metrics we employ are well-established in the green computing literature [1] and directly map to the dimensions of performance and energy efficiency we aim to compare.

5.2 Internal Validity

Internal validity refers to the degree to which the experimental design supports a causal link between the compiler version and the observed outcomes.

Single benchmark. Our initial analysis relied solely on a Mandelbrot computation. We mitigated this threat by replicating the experiment on the NPB-RUST suite, which covers eight diverse HPC kernels and applications. The consistent energy trends across all programs strengthen the internal validity. Nevertheless, all benchmarks are still CPU-bound and single-threaded; memory-bound or parallel workloads may behave differently.

Compiler configuration. We used only the `opt-level=3` flag. Other settings, such as Link-Time Optimization (LTO), target-specific instructions (`target-cpu=native`), or Profile-Guided Optimization (PGO), could alter energy consumption and change the version-vs-version comparison. Our choice reflects a common default but does not explore the full optimization space.

Version granularity. We tested 13 major stable releases, roughly one per year. The Rust compiler follows a six-week release cycle; many intermediate versions and nightly builds exist. The precise point at which the energy improvement appeared (attributed to Rust 1.32.0) is therefore approximate—the actual change may have started in a late 1.31 nightly or consolidated over several subsequent minor releases.

System noise. Although we disabled unnecessary applications, Wi-Fi, and Bluetooth, a modern Linux desktop inevitably introduces background activity (kernel threads, daemons, filesystem flushes). We mitigated this by averaging five runs per version, but a small amount of uncontrolled variance remains.

Thermal assumptions. Our protocol waited for the CPU to cool below a reference temperature plus a 3 °C variance. However, ambient room temperature was not actively controlled, which could introduce a slight systematic bias if measurements spanned significantly different environmental conditions.

5.3 External Validity

External validity concerns the generalizability of our findings to other settings, populations, and hardware.

Single hardware platform. All experiments were performed on a single laptop with an Intel Core i7-8750H and 32 GB of DDR4 RAM. The absolute energy values and possibly the relative improvements may differ on other architectures (e.g., AMD processors, ARM-based systems, or server-class CPUs with different uncore components). Notably, the dramatic reduction in GPU energy we observed applies only to the integrated Intel UHD Graphics 630; systems with discrete GPUs or different integrated graphics may not exhibit the same behavior.

5.4 Reliability

Reliability concerns the extent to which the study can be replicated by independent researchers. We have made the complete artifact available as a replication package, including the benchmark source code, measurement scripts, raw CSV

data, and analysis tools.¹ This allows others to repeat the experiment on the same or similar hardware and verify the core findings. Nevertheless, slight variations in OS kernel version, system libraries, or measurement overhead could lead to minor numerical differences; we expect the overall trends to remain robust.

Despite these threats, the internal consistency of our comparison is strong: all versions were tested on the same hardware, with the same benchmark, using identical measurement infrastructure, and the observed effect sizes (e.g., 25% energy reduction) far exceed the typical variability, lending confidence to our main conclusions.

6 Conclusion and Future Work

This paper presented a longitudinal study of energy consumption and execution time for a CPU-bound benchmark compiled across 13 versions of the Rust compiler, from the inaugural 1.0.0 release to the latest stable version. Using Intel RAPL, we measured package, core, GPU, and DRAM energy under temperature-controlled conditions, and we applied the Speedup, Greenup, and Powerup metrics to quantify compiler improvements.

Our findings demonstrate that the Rust compiler’s evolution has delivered substantial and measurable energy savings. Total package energy decreased by 25.3% and execution time improved by 4.6% from the earliest to the most recent compiler. The largest single improvement occurred at Rust 1.32.0, where a 15.6% drop in package energy coincided with a near-total elimination of GPU power draw and a 33% reduction in DRAM energy, strongly suggesting that the underlying LLVM backend upgrade (to version 7) introduced more efficient code generation patterns. After version 1.58.0, the gains plateaued, indicating diminishing returns from subsequent compiler optimizations for this particular workload.

These results have practical implications for software developers and system operators. Upgrading the Rust compiler can reduce the energy footprint of existing code by up to a quarter with no source code changes and with a negligible impact on runtime. In a cloud or data-center context where electricity cost and carbon footprint are first-order concerns, simply adopting a recent compiler version represents a low-effort, high-impact green software practice. The robustness of these findings is reinforced by the NPB-Rust experiments, which show that the compiler-driven energy savings persist across a broad range of computational patterns, with all eight programs exhibiting measurable improvements.

Future work should extend this analysis in several directions. First, the benchmark suite should be expanded to include a broader range of computational patterns running on higher/costly classes, such as memory-bound kernels (e.g., matrix multiplication), I/O-heavy workloads, and parallel

¹Available at <https://github.com/Trenion/rust-compiler-energy-study>

applications—perhaps by adopting the NPB-Rust benchmark suite [14], even though it doesn’t have a lot of compatible Rust versions. Second, the study should be replicated on additional hardware architectures, including AMD CPUs, ARM-based systems, and servers with discrete GPUs, to test the generality of the findings. Third, a finer-grained investigation using nightly builds could precisely identify the LLVM passes or Rust compiler commits responsible for the energy improvements. Finally, the interplay between compiler version and other optimization flags (LTO, PGO, target-specific tuning) should be examined to build a more complete picture of how developers can minimize the energy consumption of Rust programs.

References

- [1] Sarah Abdulsalam, Ziliang Zong, Qijun Gu, and Meikang Qiu. 2015. Using the Greenup, Powerup, and Speedup Metrics to Evaluate Software Energy Efficiency. In *Proceedings of the 2015 Sixth International Green and Sustainable Computing Conference (IGSC)*. 1–8. doi:10.1109/IGCC.2015.7393699
- [2] Alexandru Agache, Marc Brooker, Andreea Florescu, et al. 2020. Firecracker: Lightweight Virtualization for Serverless Applications. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*. 419–434.
- [3] Brian Anderson, Lars Bergstrom, et al. 2016. Experience Report: Developing the Servo Web Browser Engine Using Rust. In *Proceedings of the 2016 ACM Conference on Innovations in Theoretical Computer Science*.
- [4] R. H. Dennard, F. H. Gaensslen, H.-N. Yu, V. L. Rideout, E. Bassous, and A. R. LeBlanc. 1974. Design of Ion-Implanted MOSFET’s with Very Small Physical Dimensions. *IEEE J. Solid-State Circuits* 9, 5 (1974), 256–268. doi:10.1109/JSSC.1974.1050511
- [5] Spencer Desrochers, Chad Paradis, and Vincent M. Weaver. 2016. A Validation of DRAM RAPL Power Measurements. In *Proceedings of the International Symposium on Memory Systems (MEMSYS)*. 455–470. doi:10.1145/2989081.2989088
- [6] Felipe P. Etcheverria, Rafael R. de Matos, K. Brumati, and Lucas M. Schnorr. 2025. Dynamic Load Balancing and Scalability Analysis of the Mandelbrot Set in a Multi-Threaded HPC Application. In *Anais do XXVI Workshop de Iniciação Científica – Simpósio em Sistemas Computacionais de Alto Desempenho (SSCAD Estendido)*. 33–40. doi:10.5753/sscad_estendido.2025.15894
- [7] Mark Horowitz, Thomas Indermaur, and Ricardo Gonzalez. 1994. Low-Power Digital Design. In *Proceedings of the IEEE Symposium on Low Power Electronics*. 8–11. doi:10.1109/LPE.1994.573184
- [8] Connor Imes and powercap contributors. 2024. powercap: C bindings to the Linux Power Capping Framework in sysfs. GitHub repository. <https://github.com/powercap/powercap>
- [9] Connor Imes and raplcap contributors. 2024. raplcap: RAPL power capping C interface with multiple implementations. GitHub repository. <https://github.com/powercap/raplcap>
- [10] Connor Imes, Huazhe Zhang, Kevin Zhao, and Henry Hoffmann. 2019. CoPPER: Soft Real-Time Application Performance Using Hardware Power Capping. In *Proceedings of the 2019 IEEE International Conference on Autonomic Computing (ICAC)*. 31–41. doi:10.1109/ICAC.2019.00015
- [11] Intel Corporation. 2024. *Intel® 64 and IA-32 Architectures Software Developer’s Manual, Volume 3B: System Programming Guide*. <https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html> Chapter 14.9: Running Average Power Limit (RAPL).
- [12] lm-sensors developers. 2024. lm-sensors: Linux hardware monitoring. GitHub repository. <https://github.com/lm-sensors/lm-sensors>
- [13] Benoit B. Mandelbrot. 1980. Fractal Aspects of the Iteration of $z \rightarrow z\lambda(1 - z)$ for Complex λ and z . *Ann. N. Y. Acad. Sci.* 357, 1 (1980), 249–259. doi:10.1111/j.1749-6632.1980.tb29690.x
- [14] Eduardo M. Martins, Leonardo G. Faé, Renato B. Hoffmann, Lucas S. Bianchessi, and Dalvan Griebler. 2025. NPB-Rust: NAS Parallel Benchmarks in Rust. *arXiv preprint arXiv:2502.15536* (2025). arXiv:2502.15536 [cs.DC]
- [15] The Rust Core Team. 2015. Announcing Rust 1.0. The Rust Programming Language Blog. <https://blog.rust-lang.org/2015/05/15/Rust-1.0.html>
- [16] The Rust Core Team. 2018. Announcing Rust 1.31 and Rust 2018. The Rust Programming Language Blog. <https://blog.rust-lang.org/2018/12/06/Rust-1.31-and-rust-2018.html>
- [17] The Rust for Linux Project. 2023. Rust for Linux. <https://rust-for-linux.com>
- [18] The Rust Project Developers. 2025. Rust Release History. <https://forge.rust-lang.org/release-history.html> Accessed: 2025-05-23.
- [19] Claes Wohlin, Per Runeson, Martin Höst, Magnus C. Ohlsson, Björn Regnell, and Anders Wesslén. 2012. *Experimentation in Software Engineering*. Springer.